

Algorithms Project

Problem 5: Diagonal Difference

Solving the Diagonal Difference problem using two approaches:
Non-Recursive (Iterative) & Recursive

Problem Statement

Given an $n \times n$ square matrix, calculate the absolute difference between the sums of its two diagonals:

- Primary diagonal (top-left $\rightarrow$ bottom-right): elements where $i == j$
- Secondary diagonal (top-right $\rightarrow$ bottom-left): elements where $i + j == n - 1$

Example

For the matrix:

```
1  2  3
4  5  6
9  8  9
```

Primary diagonal = $1 + 5 + 9 = 15$
Secondary diagonal = $3 + 5 + 9 = 17$
Absolute difference = $|15 - 17| = 2$

Constraints & I/O

Item	Details
Input	Line 1: integer n (matrix size). Next n lines: n space-separated integers per row.
Constraint	$-100 < arr[i][j] < 100$ (strictly, so -99 to 99 are valid values)
Output	A single integer: the absolute difference between the two diagonal sums.

Sample

Input:

```
3
11 2 4
4 5 6
10 8 -12
```

Primary diagonal = $11 + 5 + (-12) = 4$

Secondary diagonal = $4 + 5 + 10 = 19$

Output: $|4 - 19| = 15$ ✓

First Algorithm: Non-Recursive (Iterative)

1. Pseudo Code

```
Algorithm DiagonalDifference(n, array)
{
    sum1 := 0
    sum2 := 0
    For i := 0 to n-1 step 1 do
    {
        For j := 0 to n-1 step 1 do
        {
            If i = j then
                sum1 := sum1 + array[i][j]
            If i + j = n - 1 then
                sum2 := sum2 + array[i][j]
        }
    }
    return |sum1 - sum2|
}

Algorithm Main()
{
    read n
    For i := 0 to n-1 step 1 do
    {
        For j := 0 to n-1 step 1 do
        {
            read array[i][j]
            While array[i][j] <= -100 OR array[i][j] >= 100 do
                read array[i][j]    // re-prompt on invalid input
        }
    }
    result := DiagonalDifference(n, array)
    write result
}
```

2. Implementation (C)

```
#include <stdio.h>
#include <stdlib.h>

int DiagonalDifference(int n, int arr[n][n]);
```

```
int main() {
    int n, result;
    printf("Enter array size\n");
    scanf("%d", &n);
    int array[n][n];
    printf("Enter array elements\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &array[i][j]);
            while (array[i][j] <= -100 || array[i][j] >= 100) {
                printf("Invalid input, enter a value again\n");
                scanf("%d", &array[i][j]);
            }
        }
    }
    result = DiagonalDifference(n, array);
    printf("Difference is %d\n", result);
    return 0;
}

int DiagonalDifference(int n, int array[n][n]) {
    int sum1 = 0, sum2 = 0, result;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) sum1 += array[i][j];
            if (i + j == n-1) sum2 += array[i][j];
        }
    }
    result = (sum1 > sum2) ? sum1 - sum2 : sum2 - sum1;
    return result;
}
```

3. Analysis & Complexity

Basic Operations

The two comparisons inside the inner loop ($i==j$ and $i+j==n-1$) and the conditional additions are the basic operations counted.

Step Count via Summation

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} c \\ &= c \times n \times n \\ &= c \times n^2 \end{aligned}$$

Adding constant-cost initializations and the return statement:

$$C(n) = c \cdot n^2 + k$$

Time Complexity

$$T(n) = O(n^2)$$

Case Analysis

Best Case	Average Case	Worst Case
$O(n^2)$	$O(n^2)$	$O(n^2)$

All cases are identical because the algorithm always traverses the entire $n \times n$ matrix regardless of the input values.

Second Algorithm: Recursive

1. Pseudo Code

```
Algorithm DiagonalDifference(n, i, arr)
{
    If i = n then
        return 0
    res := arr[i][i] - arr[i][n - 1 - i]
    return res + DiagonalDifference(n, i + 1, arr)
}

Algorithm Main()
{
    read n
    For i := 0 to n-1 step 1 do
    {
        For j := 0 to n-1 step 1 do
        {
            read arr[i][j]
            While arr[i][j] <= -100 OR arr[i][j] >= 100 do
                read arr[i][j]
        }
    }
    result := DiagonalDifference(n, 0, arr)
    If result < 0 then
        result := -result
    write result
}
```

2. Implementation (C)

```
#include <stdio.h>
#include <stdlib.h>

int DiagonalDifference(int n, int i, int arr[n][n]);

int main() {
    int n, result;
    printf("Enter array size\n");
    scanf("%d", &n);
    int arr[n][n];
    printf("Enter array elements\n");
```

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        scanf("%d", &arr[i][j]);
        while (arr[i][j] <= -100 || arr[i][j] >= 100) {
            printf("Invalid input, enter a value again\n");
            scanf("%d", &arr[i][j]);
        }
    }
}

result = DiagonalDifference(n, 0, arr);
if (result < 0) result = -result;
printf("Difference is %d\n", result);
return 0;
}

int DiagonalDifference(int n, int i, int arr[n][n]) {
    if (i == n) return 0;
    int res = arr[i][i] - arr[i][n - 1 - i];
    return res + DiagonalDifference(n, i + 1, arr);
}
```

3. Analysis & Complexity

Basic Operations

Each recursive call performs: two array accesses ($arr[i][i]$ and $arr[i][n-1-i]$), one subtraction, one addition, and one recursive call. These are $O(1)$ per call.

Recurrence Relation

Let $T(k)$ = cost of solving from row k to row $n-1$

```
T(n) =  $\Theta(1)$  // base case: i == n
T(k) = T(k+1) +  $\Theta(1)$  // one step + recursive call
```

Substituting (expanding from row 0):

```
T(0) = T(1) + 1
      = T(2) + 2
      = ...
      = T(n) + n
      =  $\Theta(1)$  + n
      =  $\Theta(n)$ 
```

Time Complexity

$$T(n) = O(n)$$

Case Analysis

Best Case	Average Case	Worst Case
$O(n)$	$O(n)$	$O(n)$

All cases are identical because the recursion always makes exactly n calls regardless of the input values.

Comparison of Both Algorithms

Feature	Non-Recursive	Recursive
Approach	Nested loops over full matrix	Row-by-row recursive calls
Elements Visited	All n^2 elements	Only $2n$ diagonal elements
Time Complexity	$O(n^2)$	$O(n)$
Space Complexity	$O(1)$ — no extra space	$O(n)$ — call stack depth
Number of Operations	Larger: checks all cells	Smaller: only diagonal cells
Readability	Easy to understand	Concise but needs recursion knowledge
Performance (large n)	Slower	Faster
Stack Overflow Risk	None	Possible for very large n

Summary

The recursive algorithm is more efficient in time complexity ($O(n)$ vs $O(n^2)$) because it only processes the $2n$ diagonal elements instead of all n^2 elements in the matrix. However, it uses $O(n)$ stack space due to the recursive calls, while the iterative version uses $O(1)$ extra space. For practical purposes with the given constraints, both algorithms produce correct results, but the recursive approach demonstrates a smarter traversal strategy.